

INF-14 Navrhnu a vytvořím strukturu vzájemného propojení tabulek; navrhnu procesy zpracování dat.

Na test máš 30 minut. Pracuj samostatně, bez pomoci druhých i umělé inteligence. Úlohy se týkají databáze online lekcí (žáci, učitelé, lekce, přihlášky); jména a adresy jsou smyšlené. Návrh tabulek můžeš zapsat textem nebo nakreslit (Excalidraw, papír). Program v úloze 8 řeš bez spouštění, příkaz v úloze 5 si po zapsání můžeš ověřit v SQLite. Odpovědi nahraj do portfolia na Google Disku.

1. Urči typ vztahu (1:1, 1:N, M:N): a) žák – jeho portfolio (každý žák má právě jedno); b) učitel – lekce (lekcí vede jeden učitel); c) žák – lekce, na kterou je přihlášený; d) kurz – lekce kurzu; e) podklad v portfoliu – výstup ŠVP (jeden podklad může dokládat víc výstupů); f) lekce – její záznam na Disku (nejvýš jeden).
2. Kam patří cizí klíč a jak se bude jmenovat: a) učitel – lekce; b) kurz – lekce; c) žák – lekce (přihlášky)?
3. a) Který záznam porušuje referenční integritu a proč? b) Kdo vede lekci Funkce? c) Kolik lekcí vede Horák? d) Je v pořádku, že lekce Energie má ucitel_id NULL? Jak to v návrhu tabulky zakážeš, pokud musí mít každá lekce učitele?

ucitel		lekce			
id	prijmeni	id	tema	datum	ucitel_id
1	Horák	1	Rovnice	2026-10-05	1
2	Malá	2	Pohyb	2026-10-06	2
3	Beneš	3	Sítě	2026-10-07	4
		4	Funkce	2026-10-12	1
		5	Energie	2026-10-13	NULL

4. Přihlášky na lekce se zatím ukládají do jedné ploché tabulky. Popiš dva problémy, které tato struktura způsobí. Navrhni místo ní propojené tabulky (název, atributy, primární a cizí klíče) a urči, kolik řádků bude mít každá z nich pro tato data.

zak	email	lekce	datum	ucitel
Nováková	novakova@skola.example	Rovnice	2026-10-05	Horák
Nováková	novakova@skola.example	Pohyb	2026-10-06	Malá
Svoboda	svoboda@skola.example	Rovnice	2026-10-05	Horák
Dvořák	dvorak@skola.example	Funkce	2026-10-12	Horák
Svoboda	svoboda@skola.example	Funkce	2026-10-12	Horák

5. Napiš příkaz CREATE TABLE pro vazební tabulku prihlaska z úlohy 4 s atributem datum přihlášení. Nastav primární klíč, cizí klíče a omezení tak, aby se žák nemohl na stejnou lekci přihlásit dvakrát, žádný údaj nechyběl a aby se při smazání lekce smazaly i přihlášky na ni (zápis kaskádového mazání je v nápovědě).

```
-- nápověda: cizí klíč s kaskádovým mazáním závislých řádků
-- sloupec INTEGER REFERENCES tabulka(id) ON DELETE CASCADE
```

6. Co má databáze udělat (RESTRICT – zakázat smazání, CASCADE – smazat i závislé záznamy, SET NULL – vynulovat odkaz)? Zdůvodni. a) maže se lekce, na kterou jsou přihlášky; b) maže se učitel, který vede naplánované lekce; c) z databáze se maže žák, který školu dokončil.
7. Navrhni proces „import přihlášek“: přihlášky přicházejí z formuláře jako soubor CSV (e-mail žáka, téma a datum lekce, čas odeslání) a mají se uložit do databáze z úlohy 4. Popiš kroky vstup → kontrola → uložení → zpracování → výstup a u každého kroku uveď, kterou tabulku čteš nebo měníš. Co uděláš s chybným řádkem?

8. Co program vypíše? Vysvětli, proč byl který záznam odmítnut a jakou roli má první příkaz PRAGMA.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.execute("CREATE TABLE lekce (id INTEGER PRIMARY KEY, tema TEXT NOT NULL)")
con.execute("""CREATE TABLE prihlaska (
    zak_id    INTEGER NOT NULL,
    lekce_id INTEGER NOT NULL REFERENCES lekce(id),
    PRIMARY KEY (zak_id, lekce_id)""")
con.execute("INSERT INTO lekce VALUES (1, 'Rovnice'), (2, 'Pohyb')")
for zak, lekce in [(7, 1), (7, 2), (8, 3), (7, 1)]:
    try:
        con.execute("INSERT INTO prihlaska VALUES (?, ?)", (zak, lekce))
        print(zak, lekce, "uloženo")
    except sqlite3.IntegrityError as chyba:
        print(zak, lekce, "odmítnuto:", chyba)
print(con.execute("SELECT COUNT(*) FROM prihlaska").fetchone()[0])
```

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

Klíč a řešení

1. a) 1:1; b) 1:N (jeden učitel, mnoho lekcí); c) M:N; d) 1:N; e) M:N; f) 1:1. – ověřuje: rozpoznání typu vztahu mezi entitami
2. a) do tabulky lekce: lekce.ucitel_id → ucitel.id; b) do tabulky lekce: lekce.kurz_id → kurz.id; c) do nové vazební tabulky prihlaska se dvěma cizími klíči zak_id → zak.id a lekce_id → lekce.id. – ověřuje: umístění cizího klíče u vztahů 1:N a M:N
3. a) lekce 3 (Sítě) – ucitel_id 4 odkazuje na neexistujícího učitele; b) Horák (ucitel_id 1); c) 2 (Rovnice, Funkce); d) NULL integritu neporušuje (neodkazuje nikam), ale pokud lekce musí mít učitele, je to chyba – zakáže ji omezení NOT NULL u sloupce ucitel_id (spolu s REFERENCES ucitel(id)). – ověřuje: čtení propojených tabulek a referenční integrita
4. Problémy: údaje se opakuji – změna e-mailu nebo učitele lekce se musí udělat na více místech a snadno vznikne nesoulad; lekci bez přihlášených nelze uložit; smazáním poslední přihlášky zmizí i informace o lekci. Návrh: zak(id PK, prijmeni, email UNIQUE) – 3 řádky; ucitel(id PK, prijmeni) – 2 řádky; lekce(id PK, tema, datum, ucitel_id FK → ucitel.id) – 3 řádky; prihlaska(zak_id FK → zak.id, lekce_id FK → lekce.id, PK (zak_id, lekce_id)) – 5 řádků. – ověřuje: návrh struktury propojených tabulek z nenormalizovaných dat
 1. Entity: žák, učitel, lekce; přihláška je vztah M:N mezi žákem a lekcí.
 2. Žáci: Nováková, Svoboda, Dvořák → 3 řádky, e-mail jen jednou.
 3. Učitelé: Horák, Malá → 2 řádky.
 4. Lekce: Rovnice, Pohyb, Funkce s datem a ucitel_id (vztah 1:N) → 3 řádky.
 5. Vazební tabulka prihlaska: jeden řádek za každou přihlášku → 5 řádků, klíč dvojice (zak_id, lekce_id).
5. CREATE TABLE prihlaska (zak_id INTEGER NOT NULL REFERENCES zak(id), lekce_id INTEGER NOT NULL REFERENCES lekce(id) s kaskádovým mazáním podle nápoředy, datum_prihlasi TEXT NOT NULL, PRIMARY KEY (zak_id, lekce_id)); – primární klíč z dvojice (zak_id, lekce_id) zabrání dvojí přihlášce, NOT NULL chybějícím údajům, kaskáda smaže přihlášky smazané lekce (v SQLite je nutné zapnout PRAGMA foreign_keys = ON). – ověřuje: vytvoření struktury propojení tabulek v SQL
6. a) CASCADE – přihlášky bez lekce nemají smysl (u lekcí, které už proběhly, může být lepší RESTRICT kvůli evidenci účasti); b) RESTRICT – lekce musí nejdřív dostat jiného učitele, jinak by zůstaly bez vedoucího (SET NULL jen tehdy, když NULL povolíš a lekce hned někdo převezme); c) CASCADE pro jeho přihlášky – osobní údaje nemají zůstat déle, než je třeba (případně anonymizace, pokud je potřeba statistika). – ověřuje: návrh pravidel zpracování dat při mazání propojených záznamů
7. Např.: 1) záloha databáze a načtení CSV; 2) kontrola řádku: platný e-mail, lekce s daným tématem a datem existuje (čtení lekce), žák už není přihlášen (čtení prihlaska), případně kapacita; 3) najdi žáka podle e-mailu (čtení zak), pokud neexistuje, vlož ho (vložení do zak); 4) vlož přihlášku (vložení do prihlaska); 5) zpracování: seznam přihlášených na lekci pro učitele (čtení prihlaska, zak, lekce – spojení JOIN); 6) výstup: potvrzení žákům, souhrn importu. Chybný řádek neukládat, zapsat do protokolu chyb s důvodem a ohlásit – nic neopravovat odhadem. – ověřuje: návrh procesu zpracování dat nad propojenými tabulkami
 1. Vstup: soubor CSV, jeden řádek = jedna přihláška.
 2. Kontrola dřív než zápis: formát, existence lekce, duplicita.
 3. Uložení v pořadí podle vazeb: nejdřív zak (strana 1), pak prihlaska (odkazuje na zak a lekce).
 4. Chybné řádky do protokolu, ne do databáze.
 5. Zpracování a výstup čtou spojené tabulky.
8. Vypiše: 7 1 uloženo; 7 2 uloženo; 8 3 odmítnuto: FOREIGN KEY constraint failed; 7 1 odmítnuto: UNIQUE constraint failed: prihlaska.zak_id, prihlaska.lekce_id; 2. Lekce 3 neexistuje (cizí klíč), dvojice (7, 1) už v tabulce je (primární klíč). PRAGMA foreign_keys = ON zapíná v SQLite kontrolu cizích klíčů – bez něj by se přihláška na lekci 3 uložila. – ověřuje: ověření, jak databáze hlídá propojení tabulek

Kritéria splnění výstupu

- Správně určí typ vztahu aspoň v 5 z 6 dvojic (úloha 1) a cizí klíč umístí na stranu N, u M:N do vazební tabulky (úloha 2).
- Z ploché tabulky navrhne propojené tabulky s primárními a cizími klíči, ve kterých se žádný údaj zbytečně neopakuje, a správně určí počty řádků (úloha 4).
- Zapiše funkční CREATE TABLE s cizími klíči a vhodnými omezeními a vysvětlí, jak databáze hlídá referenční integritu (úlohy 3, 5 a 8).

- Navrhne proces zpracování dat s kontrolou vstupu, pořadím zápisů podle vazeb, ošetřením chyb a zdůvodněným chováním při mazání (úlohy 6 a 7).

Pro učitele: varianty a přizpůsobení

Obměny: jiná doména – domácí knihovna, turnaj, spotřebiče v místnostech (H60), evidence měření z domácích pokusů. Praktická varianta pro portfolio: Žák navrženou databázi skutečně vytvoří v SQLite nebo v Pythonu v Colabu, vloží ukázková data a doloží dotaz se spojením tabulek. Silnějším lze přidat vztah 1:1 s omezením UNIQUE nebo vazební tabulku s vlastními atributy (položky objednávky s cenou). ER diagram lze kreslit v Excalidraw při sdílené obrazovce. Kratší verze: úlohy 1, 4 a 7.

Výstup se probírá v lekcích: 3.+4. H42 Databázová tabulka (týden 21); 3.+4. H43 Propojení tabulek (týden 22); 3.+4. H60 Elektřina doma (týden 30)